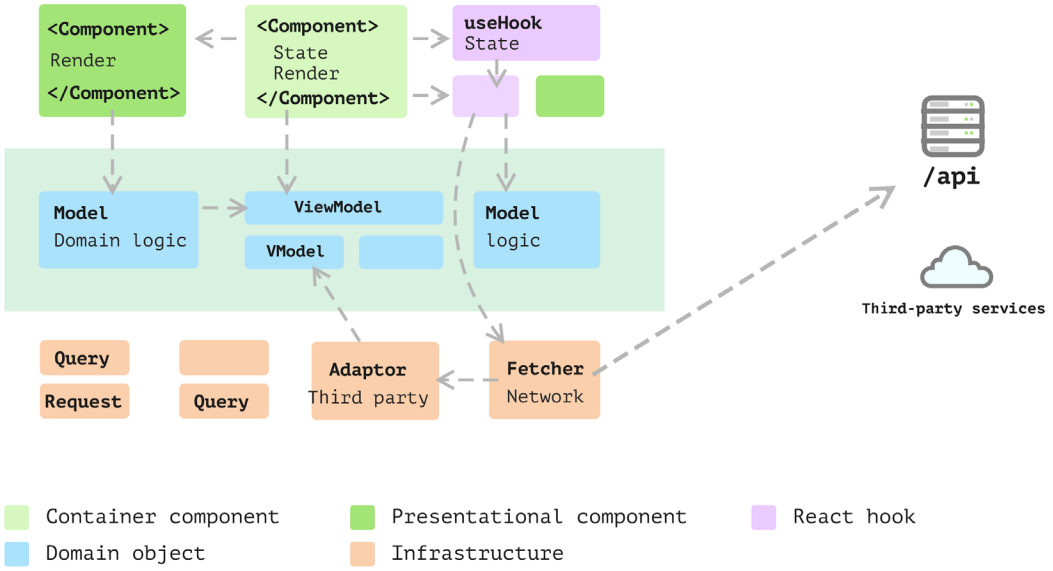

Figure 11.5: Layered frontend application

As illustrated in *Figure 11.5*, we can allocate different parts to separate folders, each distinctly and physically isolated from the others. This way, if there's a need to modify models, you won't need to navigate through the views folder, and vice versa.

That was a high-level overview of the evolution process, and you should have a taste of how you should structure your code or at least what the direction should be. In larger-scale applications, you'll likely encounter a variety of modules and functions, each tailored to different aspects of the app. This could include a request module for handling network requests, or adapters designed to interface with various data vendors, such as Google's login API or payment gateway clients.

However, there will be many details, such as how to define a model, how to access a model from views or Hooks, and so on. You need to consider this before applying the theory to your application.

Read more

You can find a detailed explanation of Presentation Domain Data Layering here: `https://martinfowler.com/bliki/PresentationDomainDataLayering.html`.

In the following sections, I'll guide you through expanding the Code Oven application we introduced in *Chapter 7*, to showcase essential patterns and design principles for large frontend applications.

Enhancing the Code Oven application

Recall that at the end of *Chapter 7*, we developed the basic structure of a pizza store application named Code Oven, leveraging test-driven development to establish a solid foundation for the app.

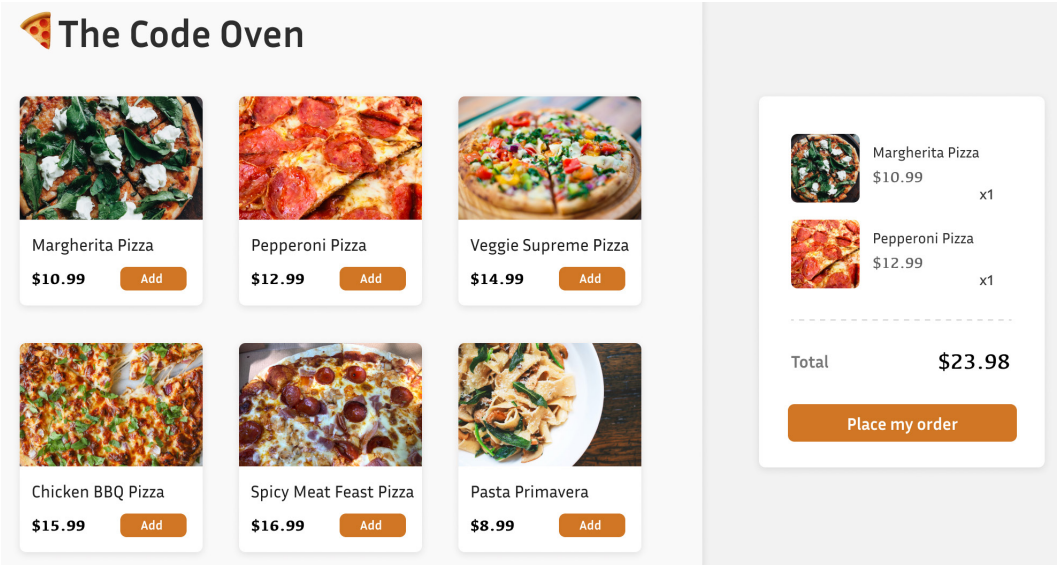

Figure 11.6: The Code Oven application

> **Note**
>
> Remember that we employed the design mockup as a guide, not to implement all the details exhaustively. The primary goal remains to illustrate how to refactor the code while preserving its maintainability.

Although we didn't delve much into feature implementation in that chapter, in this chapter, we'll extend our setup further. We'll explore how different architectural types can assist us in managing complexity.

As a refresher, by the end of *Chapter 7*, our structure looked like this:

```
export function PizzaShopApp() {
  const [cartItems, setCartItems] = useState<string[]>([]);

  const addItem = (item: string) => {
    setCartItems([...cartItems, item]);
  };

  return (
```

```
    <>
      <h1>The Code Oven</h1>
      <MenuList onAddMenuItem={addItem} />
      <ShoppingCart cartItems={cartItems} />
    </>
  );
}
```

And we assumed the data to be in this shape:

```
const pizzas = [
  "Margherita Pizza",
  "Pepperoni Pizza",
  "Veggie Supreme Pizza"
];
```

While this setup allows consumers to browse what the restaurant offers, it would be significantly more useful if we enabled online ordering. However, one immediate issue is that the pizzas lack prices and descriptions, crucial for supporting online orders. Descriptions are also vital as they list the ingredients, informing consumers of what's included.

Saying that, it's actually not practical to define menu data within the JavaScript code. Typically, we'd have a service hosting such data, providing more detailed information.

To show this, suppose we have data hosted on the `https://api.code-oven.com/menus` remote service, defined as follows:

```
[
  {
    "id": "p1",
    "name": "Margherita Pizza",
    "price": 10.99,
    "description": "Classic pizza with tomato sauce and mozzarella",
    "ingredients": ["Tomato Sauce", "Mozzarella Cheese", "Basil",
     "Olive Oil"],
    "allergyTags": ["Dairy"],
    "calories": 250,
    "category": "Pizza"
  },
  //...
]
```

To bridge the gap between our app and this data, we need to define a type for the remote data, like so:

```
type RemoteMenuItem = {
  id: string;
```

```
  name: string;
  price: number;
  description: string;
  ingredients: string[];
  allergyTags: string[];
  category: string;
  calories: number
}
```

Now, to integrate this remote menu data, we'll utilize `useEffect` to fetch the data, and then display the items once fetched. We'll make these changes within the `MenuList` component:

```
const MenuList = ({
  onAddMenuItem,
}: {
  onAddMenuItem: (item: string) => void;
}) => {
  const [menuItems, setMenuItems] = useState<string[]>([]);

  useEffect(() => {
    const fetchMenuItems = async () => {
      const result = await fetch('https://api.code-oven.com/menus');
      const menuItems = await result.json();

      setMenuItems(menuItems.map((item: RemoteMenuItem) => item.
       name));
    }

    fetchMenuItems();
  }, [])

  return (
    <div data-testid="menu-list">
      <ol>
        {menuItems.map((item) => (
          <li key={item}>
            {item}
            <button onClick={() => onAddMenuItem(item)}>Add</button>
          </li>
        ))}
      </ol>
    </div>
  );
};
```

Here, the `MenuList` component fetches a list of menu items from an external API upon the initial render and displays this list. Each item comes with an **Add** button, and clicking this button triggers the `onAddMenuItem` function, passed as a prop to `MenuList`, with the item name as its argument.

By mapping `RemoteMenuItem` to a string after fetching the data, we ensure our tests continue to pass.

Now, we aim to reveal the price and display the ingredients from the data to the UI components. However, given the potentially long list of ingredients, we'll only show the first three to avoid occupying too much screen space. Also, we want to use lowercase `category` and rename it to `type`.

Initially, we define a new type to better structure our data:

```
type MenuItem = {
  id: string;
  name: string;
  price: number;
  ingredients: string[];
  type: string;
}
```

Here, the `MenuItem` type includes the item's `id`, `name`, `price`, `ingredients`, and `type` properties.

Now, it's time to update our `MenuList` component to use this new type:

```
const MenuList = ({
  onAddMenuItem,
}: {
  onAddMenuItem: (item: string) => void;
}) => {
  const [menuItems, setMenuItems] = useState<MenuItem[]>([]);

  useEffect(() => {
    const fetchMenuItems = async () => {
      const result = await fetch("http://api.code-oven.com/menus");
      const menuItems = await result.json();

      setMenuItems(
        menuItems.map((item: RemoteMenuItem) => {
          return {
            id: item.id,
            name: item.name,
            price: item.price,
            type: item.category.toUpperCase(),
            ingredients: item.ingredients.slice(0, 3),
          };
        })
```

```
      );
    };

    fetchMenuItems();
  }, []);

  return (
    <div data-testid="menu-list">
      <ol>
        {menuItems.map((item) => (
          <li key={item.id}>
            <h3>{item.name}</h3>
            <span>${item.price}</span>
            <div>
              {item.ingredients.map((ingredient) => (
                <span>{ingredient}</span>
              ))}
            </div>
            <button onClick={() => onAddMenuItem(item.name)}>Add
             </button>
          </li>
        ))}
      </ol>
    </div>
  );
};
```

In the `MenuList` component, we've now made use of the `MenuItem` type in our `useState` Hook. The `fetchMenuItems` function, triggered within `useEffect`, reaches out to the API, fetches the menu items, and maps over them to transform the data into the desired `MenuItem` format. This transformation includes retaining only the first three items from the `ingredients` array for each item.

Each `MenuItem` component is then rendered as a list item within the component. We display the item's name, price, and iterate over the `ingredients` array to render each ingredient.

While the code is functional, there's a concern: we've intertwined network requests, data mapping, and rendering logic within a single component. It's a sound practice to separate view-related code from non-view code, ensuring cleaner, more maintainable code.

Refactoring the MenuList through a custom Hook

We're no strangers to using custom Hooks for data fetching – it's a practice that enhances readability and organizes logic neatly. In our scenario, extracting the `menuItems` state and the fetching logic into a separate Hook will declutter the `MenuList` component.

So, let's create a Hook named `useMenuItems`:

```
const useMenuItems = () => {
  const [menuItems, setMenuItems] = useState<MenuItem[]>([]);

  useEffect(() => {
    const fetchMenuItems = async () => {
      const result = await fetch(
        "https://api.code-oven.com/menus"
      );
      const menuItems = await result.json();

      setMenuItems(
        menuItems.map((item: RemoteMenuItem) => {
          // ... transform RemoteMenuItem to MenuItem
        })
      );
    };

    fetchMenuItems();
  }, []);

  return { menuItems };
};
```

Within the `useMenuItems` Hook, we initialize the `menuItems` state with an empty array. When the Hook mounts, it triggers the `fetchMenuItems` function that fetches data from the specified URL. Following the fetch, a mapping operation is performed to convert each `RemoteMenuItem` object to a `MenuItem` object. The transformation details are omitted here, but it's where we adapt the fetched data to the desired format. Subsequently, the transformed menu items are set to the `menuItems` state.

Now, in our `MenuList` component, we can simply call `useMenuItems` to obtain the `menuItems` array:

```
const MenuList = ({
  onAddMenuItem,
}: {
  onAddMenuItem: (item: string) => void;
}) => {
  const { menuItems } = useMenuItems();
  //...
}
```

This refactoring is quite beneficial, redirecting `MenuList` back to a streamlined state and reinstating its single responsibility. However, when we shift our focus to the `useMenuItems` Hook, particularly the data mapping segment, a few operations occur. It fetches data and trims off some unused fields such as `description` and `calories` from the remote data. It also encapsulates the logic to retain only the first three ingredients. Ideally, we'd like to centralize this transformation logic into a common location, ensuring a tidy and manageable code structure.

Transitioning to a class-based model

As touched upon in *Chapter 8*, employing the **Anti-Corruption Layer** (**ACL**) pattern can be a strategic move for managing our data effectively. We'll employ classes in TypeScript to encapsulate data and logic in a unified location, referred to as a model. A significant step in this direction would be transitioning our `MenuItem` type definition into a class, hence centralizing all mapping logic within this class. This setup will serve as a dedicated hub for any future data shape alterations and related logic.

Transitioning `MenuItem` from a type to a class is straightforward. We require a constructor to accept `RemoteMenuItem` and some getter functions to access the data:

```
export class MenuItem {
  private readonly _id: string;
  private readonly _name: string;
  private readonly _type: string;
  private readonly _price: number;
  private readonly _ingredients: string[];

  constructor(item: RemoteMenuItem) {
    this._id = item.id;
    this._name = item.name;
    this._price = item.price;
    this._type = item.category;
    this._ingredients = item.ingredients;
  }

  // ... getter functions for id, name, price just returns the private
         fields

  get type() {
    return this._type.toLowerCase();
  }

  get ingredients() {
    return this._ingredients.slice(0, 3);
  }
}
```

In the `MenuItem` class, we define private `readonly` properties for `id`, `name`, `type`, `price`, and `ingredients`. The constructor initializes these properties using values from a `RemoteMenuItem` object passed to it. We then have getter methods for each property to provide read-only access to their values. Particularly, the `ingredients` getter returns only the first three items from the `ingredients` array.

Though, at a glance, this setup seems to have more code compared to a simple type definition, it effectively encapsulates the data and exposes it in a controlled manner. This aligns with the principles of immutability and encapsulation. The class structure's beauty is its capability to house behaviors – in our case, the slicing logic for ingredients is tucked neatly within the class.

With this new class in place, our `useMenuItems` Hook becomes more streamlined:

```
export const useMenuItems = () => {
  //...

  useEffect(() => {
    const fetchMenuItems = async () => {
      //...
      setMenuItems(
        menuItems.map((item: RemoteMenuItem) => {
          return new MenuItem(item);
        })
      );
    };

    fetchMenuItems();
  }, []);

  return { menuItems };
};
```

Now, the `useMenuItems` Hook merely maps over the fetched menu items, creating a new instance of `MenuItem` for each, which significantly tidies up the transformation logic previously housed within the Hook.

The benefits of a class-based model

Transitioning to a class-based model from a simple type comes with a set of advantages that could serve our application well in the long run:

- **Encapsulation**: A class brings related properties and methods under one roof, thus promoting clear structure and organization. It also restricts direct data access, fostering better control and data integrity.

- **Method behavior**: For complex behaviors or operations associated with a menu item, a class provides a structured platform to define these methods, whether they relate to data manipulation or other business logic.
- **Inheritance and polymorphism**: In the case of a hierarchy or polymorphic behavior among menu items, a class structure is indispensable. It allows different menu item types to inherit from a common base class, overriding or extending behavior as needed.
- **Consistent interface**: Classes ensure a consistent interface to the data, which is invaluable when multiple application parts interact with menu items.
- **Read-only properties**: Classes enable the definition of read-only properties, thereby controlling data mutation. This is a crucial aspect of maintaining data integrity and working with immutable data structures.

Now, as we transition into expanding our application's functionality with a shopping cart, it's crucial to approach this new section with the lessons learned from our data modeling exercise. This will ensure a structured and effective implementation, paving the way for a user-friendly online ordering experience.

Implementing the ShoppingCart component

As we venture into the implementation of the `ShoppingCart` component, we will aim to provide a seamless interface for users to review their selected items before proceeding to the checkout. Besides displaying the items, we also intend to reward our customers with some appealing discount policies.

In *Chapter 7*, we defined a rudimentary `ShoppingCart` component, as shown here:

```
export const ShoppingCart = ({ cartItems }: { cartItems: string[] })
=> {
  return (
    <div data-testid="shopping-cart">
      <ol>
        {cartItems.map((item) => (
          <li key={item}>{item}</li>
        ))}
      </ol>
      <button disabled={cartItems.length === 0}>Place My Order
       </button>
    </div>
  );
};
```

The `ShoppingCart` component accepts a `cartItems` prop, which is an array of strings. It returns a `div` tag containing an ordered list (`<ol>`), where each item in the `cartItems` array is rendered as a list item (`<li>`). Below the list, a **Place My Order** button is rendered, which is disabled if the `cartItems` array is empty.

However, to enhance the user experience, it's crucial to display the price for each item and the total amount beneath the item list, yet above the **Place My Order** button. Here's how we can augment our component to fulfill these requirements:

```
export const ShoppingCart = ({ cartItems }: { cartItems: MenuItem[] })
=> {
  const totalPrice = cartItems.reduce((acc, item) => (acc += item.
price), 0);

  return (
    <div data-testid="shopping-cart" className="shopping-cart">
      <ol>
        {cartItems.map((item) => (
          <li key={item.id}>
            <h3>{item.name}</h3>
            <span>${item.price}</span>
          </li>
        ))}
      </ol>
      <div>Total: ${totalPrice}</div>
      <button disabled={cartItems.length === 0}>Place My Order
       </button>
    </div>
  );
};
```

The `ShoppingCart` component is now equipped to accept a `cartItems` prop, which comprises an array of `MenuItem` objects (instead of a simple string). To compute the total price of items in the cart, we employ the `reduce` method. This method iterates over each item, accumulating their prices to present a total. The component then returns a JSX markup that renders a list of cart items, each displaying its name and price.

This revamped `ShoppingCart` component not only enhances the clarity of the order for users but also lays down a foundation for introducing discount policies, which we can explore as we continue refining our application.

Applying discounts to Items

Let's assume we have different discount policies for different types of menu items. For instance, pizzas with more than three toppings receive a 10 percent discount, while large pasta dishes enjoy a 15 percent discount.

To incorporate this, we initially attempt to extend the `MenuItem` class with a new field called `calculateDiscount`:

```
export class MenuItem {
  //... the private fields

  constructor(item: RemoteMenuItem) {
    //... assignment
  }

  get calculateDiscount() {
    return this.type === 'pizza' && this.toppings >= 3 ? this.price *
     0.1 : 0;
  }
}
```

However, we encounter a problem – since pasta dishes don't have toppings, this leads to a type error.

To resolve this, we first extract an interface named `IMenuItem`, and then have `PizzaMenuItem` and `PastaMenuItem` classes implement this interface:

```
export interface IMenuItem {
  id: string;
  name: string;
  type: string;
  price: number;
  ingredients: string[];

  calculateDiscount(): number;
}
```

Next, we define an abstract class to implement the interface, allowing `PizzaMenuItem` and `PastaMenuItem` to extend the abstract class respectively:

```
export abstract class AbstractMenuItem implements IMenuItem {
  private readonly _id: string;
  private readonly _name: string;
  private readonly _price: number;
```

```
  private readonly _ingredients: string[];

  protected constructor(item: RemoteMenuItem) {
    this._id = item.id;
    this._name = item.name;
    this._price = item.price;
    this._ingredients = item.ingredients;
  }

  static from(item: IMenuItem): RemoteMenuItem {
    return {
      id: item.id,
      name: item.name,
      price: item.price,
      category: item.type,
      ingredients: item.ingredients,
    };
  }

  //... the getter functions

  abstract calculateDiscount(): number;
}
```

In the `AbstractMenuItem` class, we introduced a static `from` method. This method takes an `IMenuItem` instance and transforms it into a `RemoteMenuItem` instance, preserving the necessary fields for our application.

The `calculateDiscount` method is declared as an abstract method, requiring its child classes to implement the actual discount calculation.

> **Note**
>
> An **abstract class** serves as a base class for other classes and cannot be instantiated on its own. It's a way to define a common interface and/or implementation for a set of derived classes. Abstract classes often contain abstract methods, which are declared without implementations, leaving it to derived classes to provide specific implementations. By doing so, abstract classes enable a common structure while ensuring that certain methods are implemented in derived classes, promoting a consistent behavior across all derived types. They are a key feature in object-oriented programming, supporting polymorphism and encapsulation.

We need to override and put the actual `calculateDiscount` logic in sub-classes. For `PizzaMenuItem`, it simply extends `AbstractMenuItem` and implements `calculateDiscount`:

```
export class PizzaMenuItem extends AbstractMenuItem {
  private readonly toppings: number;

  constructor(item: RemoteMenuItem, toppings: number) {
    super(item);
    this.toppings = toppings;
  }

  calculateDiscount(): number {
    return this.toppings >= 3 ? this.price * 0.1 : 0;
  }
}
```

The `PizzaMenuItem` class extends `AbstractMenuItem`, inheriting its properties and methods. It defines a private `readonly` property, `toppings`, to hold the number of toppings. In the constructor, it takes two arguments: `RemoteMenuItem` and `toppings` (which indicates the number of toppings). It calls the constructor of `AbstractMenuItem` with `item` using `super(item)` and initializes `this.toppings` with the passed-in `toppings` argument.

The `calculateDiscount` method is implemented to return a 10% discount if the number of toppings is 3 or more. This method overrides the abstract `calculateDiscount` method from `AbstractMenuItem`.

Similarly, we can create a `PastaMenuItem` class like so:

```
export class PastaItem extends AbstractMenuItem {
  private readonly servingSize: string;

  constructor(item: RemoteMenuItem, servingSize: string) {
    super(item);
    this.servingSize = servingSize;
  }

  calculateDiscount(): number {
    return this.servingSize === "large" ? this.price * 0.15 : 0;
  }
}
```

The relationship of these classes can be visualized as in *Figure 11.7*:

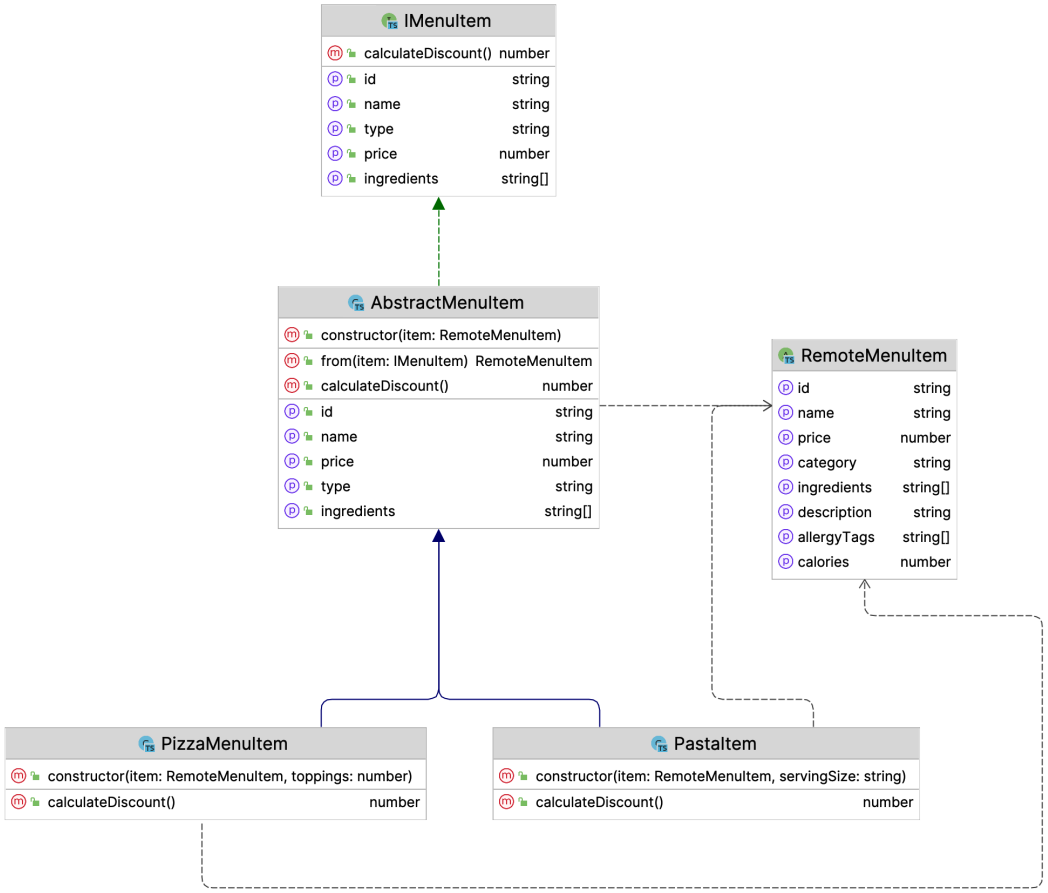

Figure 11.7: Model classes

The `AbstractMenuItem` abstract class implements the `IMenuItem` interface and uses `RemoteMenuItem`. Both `PizzaItem` and `PastaItem` are extending `AbstractMenuItem` and have their own logic for calculating the discount.

Next, in the `MenuList` component, when adding items to the shopping cart, we create instances of the right class based on the item type:

```
export const MenuList = ({}) => {
  //...
  const [toppings, setToppings] = useState([]);
  const [size, setSize] = useState<string>("small");

  const handleAddMenuItem = (item: IMenuItem) => {
```

```
    const remoteItem = AbstractMenuItem.from(item);
    if (item.type === "pizza") {
      onAddMenuItem(new PizzaMenuItem(remoteItem, toppings.length));
    } else if (item.type === "pasta") {
      onAddMenuItem(new PastaItem(remoteItem, size));
    } else {
      onAddMenuItem(item);
    }
  };

  return (
    //...
  );
};
```

The `handleAddMenuItem` function transforms the `IMenuItem` object item into a `RemoteMenuItem` object using the `AbstractMenuItem.from(item)` method. Following this, it checks the type property of the item to determine whether it's a pizza or pasta. If it's a pizza, a new `PizzaMenuItem` instance is created using `remoteItem` and the selected number of toppings, and this new item is added to the cart via the `onAddMenuItem` function. If the item is neither a pizza nor pasta, the original item is added to the cart directly through the `onAddMenuItem` function.

Lastly, within the `ShoppingCart` component, we calculate the total discount value similarly to how we calculated the total price, and use it for rendering:

```
export const ShoppingCart = ({ cartItems }: { cartItems: IMenuItem[]
}) => {
  const totalPrice = cartItems.reduce((acc, item) => (acc += item.
price), 0);
  const totalDiscount = cartItems.reduce(
    (acc, item) => (acc += item.calculateDiscount()),
    0
  );

  return (
    <div data-testid="shopping-cart">
      {/* rendering the list */}
      <div>Total Discount: ${totalDiscount}</div>
      <div>Total: ${totalPrice - totalDiscount}</div>
      <button disabled={cartItems.length === 0}>Place My Order
       </button>
    </div>
  );
};
```

The `ShoppingCart` component calculates `totalPrice` by iterating over the `cartItems` array and summing up the price of each item. Similarly, it calculates `totalDiscount` by summing up the discounts for each item, obtained by calling the `calculateDiscount()` method on each item. In the returned JSX, it renders a list and displays `totalDiscount` and the final total price (which is `totalPrice` minus `totalDiscount`) below the list.

At this juncture, the function operates effectively. Nonetheless, there are several factors to contemplate – the discount is currently specified on each product: for instance, pizza has its own discount rule while pasta has its own. What would be our approach if we need to implement a store-wide discount, such as a discount for a public holiday?

Exploring the Strategy pattern

Suppose it's a bustling Friday night, and we wish to offer a special discount on all pizzas and drinks. However, we don't intend to apply additional discounts on items already discounted — for instance, a pizza with four toppings should only receive this specific special discount.

Handling such arbitrary discounts can be complex, necessitating a decoupling of the calculation logic from the item type. Moreover, we'd like the flexibility to remove these discounts after Friday or after a certain period.

We can use a design pattern called the **Strategy pattern** to achieve flexibility here. The Strategy pattern is a behavioral design pattern that enables selecting an algorithm's implementation at runtime. It encapsulates a family of algorithms and makes them interchangeable, allowing the client to choose the most suitable one without modifying the code.

We'll extract the logic into a separate entity, defining a strategy interface as follows:

```
export interface IDiscountStrategy {
  calculate(price: number): number;
}
```

This interface provides a blueprint for different discount strategies. For example, we could have a strategy with no discount:

```
class NoDiscountStrategy implements IDiscountStrategy {
  calculate(price: number): number {
    return 0;
  }
}
```

The `NoDiscountStrategy` class implements the `IDiscountStrategy` interface with a `calculate` method that takes a price as input and returns zero, meaning no discount is applied.

And for `SpecialDiscountStrategy` component, a special discount strategy offering a 15% discount will be applied:

```
class SpecialDiscountStrategy implements IDiscountStrategy {
  calculate(price: number): number {
    return price * 0.15;
  }
}
```

To utilize these strategies, we need to slightly modify the `IMenuItem` interface:

```
export interface IMenuItem {
  // ... other fields
  discountStrategy: IDiscountStrategy;
}
```

We added `discountStrategy` with type `IDiscountStrategy` in the `IMenuItem` interface. And because we moved the logic of calculating discount into strategy, we don't need the `calculateDiscount` abstract method in `AbstractMenuItem` anymore and the class will no longer remain abstract, so we renamed it to `BaseMenuItem` instead. Instead, it will incorporate a setter for the discount strategy and implement the discount calculation:

```
export class BaseMenuItem implements IMenuItem {
  // ... other fields
  private _discountStrategy: IDiscountStrategy;

  constructor(item: RemoteMenuItem) {
    // ... other fields
    this._discountStrategy = new NoDiscountStrategy();
  }

  // ... other getters

  set discountStrategy(strategy: IDiscountStrategy) {
    this._discountStrategy = strategy;
  }

  calculateDiscount() {
    return this._discountStrategy.calculate(this.price);
  }
}
```

The `BaseMenuItem` class now implements the `IMenuItem` interface and encapsulates a discount strategy, initially set to `NoDiscountStrategy`. It defines a setter to update the discount strategy, and a `calculateDiscount` method, which delegates the discount calculation to the encapsulated discount strategy's `calculate` method, passing the item's price as an argument.

Figure 11.8 should now give you a much clearer idea of what the relationships are:

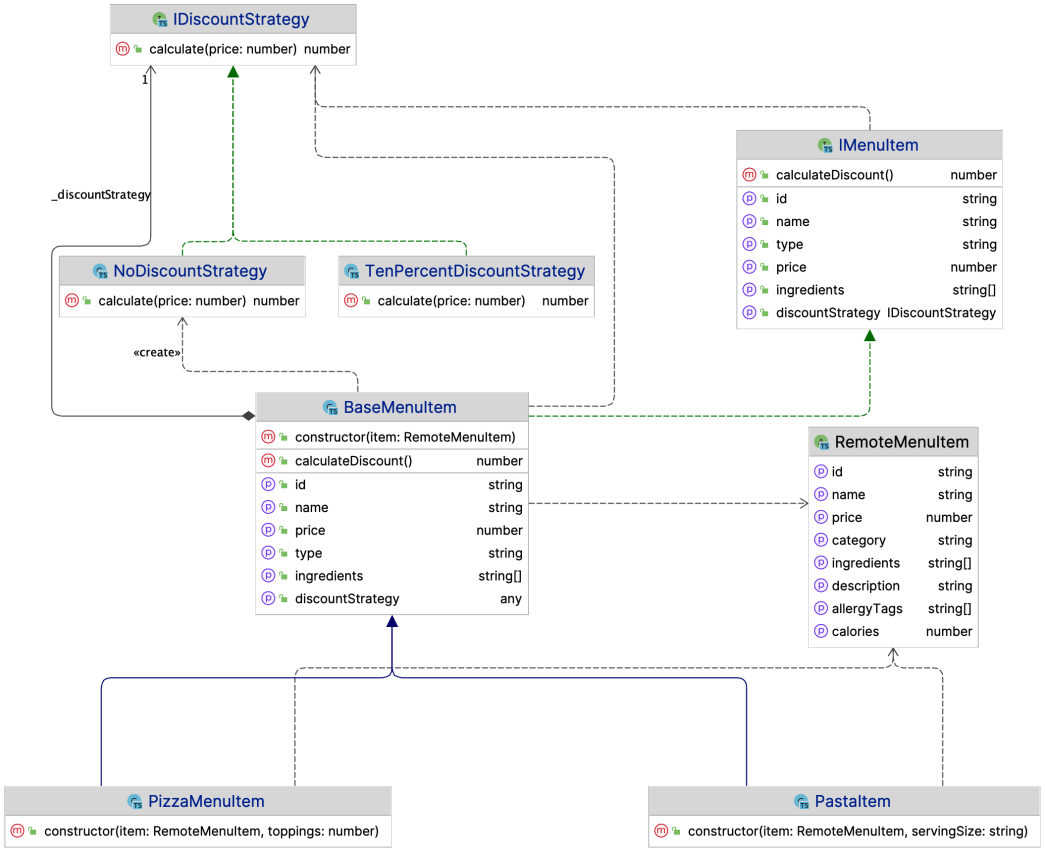

Figure 11.8: The class diagram of all the classes

As observed, `BaseMenuItem` implements the `IMenuItem` interface and utilizes `IDiscountStrategy`. There are multiple implementations of the `IDiscountStrategy` interface for specific discount algorithms, and several classes extend the `BaseMenuItem` class.

Note that the `RemoteMenuItem` type is used by all the classes implementing the `IMenuItem` interface.

Now, when we need to apply a particular strategy, it can be done effortlessly, like so:

```
export const MenuList = ({
  onAddMenuItem,
}: {
  onAddMenuItem: (item: IMenuItem) => void;
}) => {
  // ...
  const handleAddMenuItem = (item: IMenuItem) => {
    if (isTodayFriday()) {
      item.discountStrategy = new SpecialDiscountStrategy();
    }

    onAddMenuItem(item);
  };
```

In the `MenuList` component, the `handleAddMenuItem` function checks if today is Friday using the `isTodayFriday` function. If it is, it sets `discountStrategy` of the item to a new instance of `SpecialDiscountStrategy` before passing the item to the `onAddMenuItem` function, which is received as a prop. This way, a special discount is applied to the menu item on Fridays.

This setup grants us the desired flexibility. For instance, in the `handleAddMenuItem` function, depending on whether it's Friday or the item is a pizza, we can easily switch the discount strategy:

```
const handleAddMenuItem = (item: IMenuItem) => {
  if (isTodayFriday()) {
    item.discountStrategy = new SpecialDiscountStrategy();
  }

  if(item.type === 'pizza') {
    item.discountStrategy = new PizzaDiscountStrategy();
  }

  onAddMenuItem(item);
};
```

In this `handleAddMenuItem` function, depending on certain conditions, a different discount strategy is applied to the item before it's passed to the `onAddMenuItem` function. Initially, it checks if today is Friday using `isTodayFriday()` and, if true, it assigns a new instance of `SpecialDiscountStrategy` to `item.discountStrategy`. However, if the item is of type `pizza`, irrespective of the day, it overwrites `item.discountStrategy` with a new instance of `PizzaDiscountStrategy`.

This approach keeps our discount logic modular and easy to adjust, catering to different scenarios with minimal code modification. As we're extracting new logic components – Hooks, data models, domain logic (discount strategies), and views – out of the application code, it's evolving into a layered frontend application.

Delving into layered architecture

Our application has transitioned wonderfully to a more robust state, with clear, understandable, and modifiable logic, which is now also more test-friendly.

A further refinement I envision is relocating the logic present in `ShoppingCart` to a custom Hook. We can do this like so:

```
export const useShoppingCart = (items: IMenuItem[]) => {
  const totalPrice = useMemo(
    () => items.reduce((acc, item) => (acc += item.price), 0),
    [items]
  );

  const totalDiscount = useMemo(
    () => items.reduce((acc, item) => (acc += item.
     calculateDiscount()), 0),
    [items]
  );

  return {
    totalPrice,
    totalDiscount,
  };
};
```

The `useShoppingCart` Hook accepts an array of `IMenuItem` objects and computes two values – `totalPrice` and `totalDiscount`:

- `totalPrice` is calculated by reducing the items, summing up their `price` property
- `totalDiscount` is calculated by reducing the items, summing up the discount for each item obtained by calling `item.calculateDiscount()`

Both calculations are wrapped in `useMemo` to ensure they are only recomputed when the items array changes.

With this modification, `ShoppingCart` becomes elegantly simplified and can easily utilize these values:

```
export const ShoppingCart = ({ cartItems }: { cartItems: IMenuItem[]
}) => {
  const { totalPrice, totalDiscount } = useShoppingCart(cartItems);

  return (
    {/* JSX for the rendering logic */}
  );
};
```

An alternative approach would be employing the context and `useReducer` Hook to manage all logic within context and Hooks, however, since we explored that in *Chapter 8*, I'll leave further exploration to you (you can use both code examples provided in *Chapter 8* and this chapter and try to use `context` and `useReducer` to simplify `ShoppingCart`).

The layered structure of the application

We've delved into organizing components and models into separate files; it's equally vital to continue refining our project structure. Functions with distinct responsibilities should reside in different folders, streamlining navigation through the application and saving time. Our application now exhibits a fresh structural anatomy:

```
src
├── App.tsx
├── hooks
|   ├── useMenuItems.ts
|   └── useShoppingCart.ts
├── models
|   ├── BaseMenuItem.ts
|   ├── IMenuItem.ts
|   ├── PastaItem.ts
|   ├── PizzaMenuItem.ts
|   ├── RemoteMenuItem.ts
|   └── strategy
|       ├── IDiscountStrategy.ts
|       ├── NoDiscountStrategy.ts
|       ├── SpecialDiscountStrategy.ts
|       └── TenPercentageDiscountStrategy.ts
└── views
    ├── MenuList.tsx
    └── ShoppingCart.tsx
```

And that's how the layers formed. Within the view layer, we have primarily pure TSX rendering straightforward tags. These views leverage Hooks for state and side effect management. Meanwhile, in the model layer, model objects encompass business logic, algorithms for toggling between different discount strategies, and data shape transformations, among other functionalities. This structure promotes separation of concerns, making the code organized, reusable, and easier to maintain.

It's important to note the one-directional link here; the upper layer accesses the lower layer, but not vice versa. TSX uses Hooks for state management, and Hooks employ models for calculations. However, we can't use JSX or Hooks in the model layer. This layering technique facilitates change or replacement in the underlying layers without impacting the upper layers, promoting a clean and maintainable structure.

In our Code Oven application, as illustrated in *Figure 11.9*, the layout features a menu items list on the left and a shopping cart on the right. Within the shopping cart, each item displays detailed discount and price information on the page.

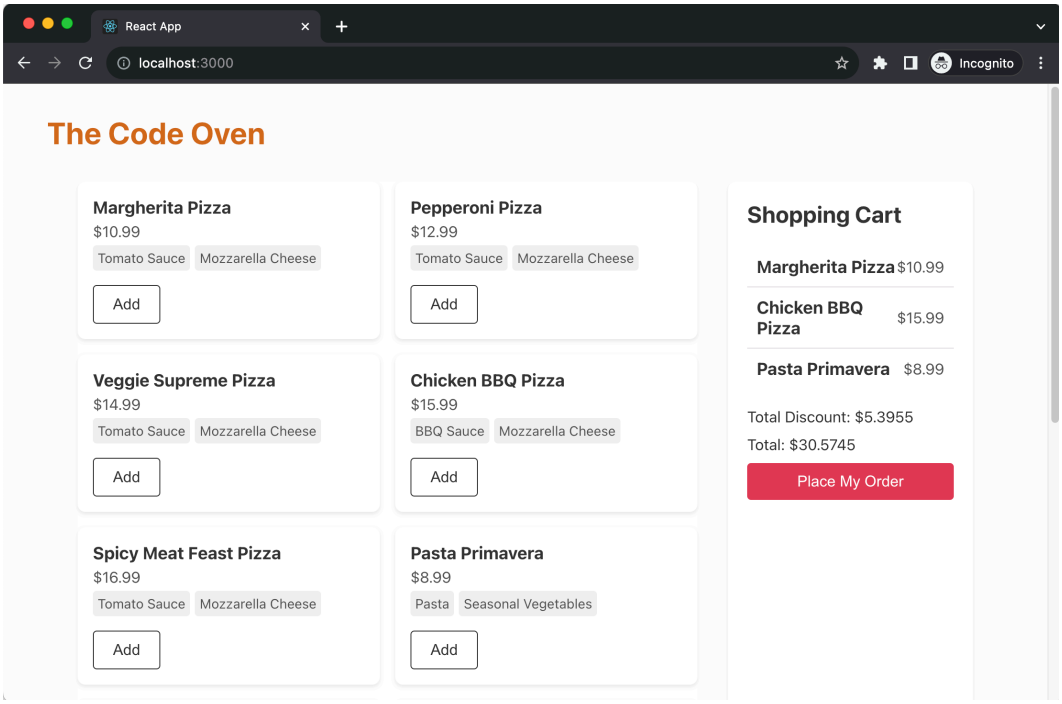

Figure 11.9: Final look and feel of the application

Advantages of layered architecture

The layered architecture confers numerous benefits:

- **Enhanced maintainability**: The division of a component into distinct segments facilitates easier identification and rectification of defects in specific code sections, thus minimizing time spent and reducing the likelihood of engendering new bugs during modifications.
- **Increased modularity**: This architecture is inherently more modular, promoting code reuse and simplifying the addition of new features. Even within each layer, such as views, the code tends to be more composable.
- **Enhanced readability**: The logic within the code becomes significantly more understandable and navigable, an asset not only for the original developer but also for others who may interact with the code base. This clarity is central to effecting changes in the code.
- **Improved scalability**: The reduced complexity within each module renders the application more scalable, making it easier to introduce new features or alterations without impacting the entire system—a critical advantage for large, complex applications projected to evolve over time.
- **Tech-stack migration**: Albeit unlikely in most projects, should the need arise, the view layer can be replaced without altering the underlying models and logic, thanks to the encapsulation of domain logic in pure JavaScript (or TypeScript) code, oblivious to the views' existence.

Summary

In this chapter, we implemented layered architecture in our application, enhancing its maintainability, modularity, readability, scalability, and potential for tech-stack migration. By segregating logic, refining the `ShoppingCart` component through a custom Hook, and organizing the application into distinct layers, we've significantly bolstered the code's structure and ease of management. This architectural approach not only streamlines the current code base but also lays a solid foundation for future expansions and refinements.

In the next chapter, we'll look at the end-to-end journey of implementing an application from scratch, using the user acceptance test-driven development approach, doing refactoring, cleaning up along the way, and always keeping our code as clean as we can.

12
Implementing an End-To-End Project

In the previous chapters, we delved into a variety of topics, including testing, **test-driven development** (**TDD**), design patterns, and design principles. These concepts are invaluable as they pave the way toward a more resilient and maintainable codebase. Now, I'd like to embark on a journey of constructing an application from the ground up, applying the knowledge we've acquired to tackle an end-to-end scenario.

The aim is to illustrate how we can dissect requirements into actionable tasks, and subsequently test and implement them. We'll also explore how to stub network requests, thereby eliminating dependencies on remote systems during development, and how to confidently refactor code without fear of breaking existing functionality.

We'll venture into building a functional weather application from scratch, interfacing with a genuine weather API server to fetch and display a list of weather data. Along the way, we'll implement accessibility features such as keyboard interactions, revisit the **Anti-Corruption Layer** (**ACL**) and single responsibility principle, and much more.

The overarching goal is to showcase the end-to-end process of crafting a functioning software solution, all while keeping the code in a maintainable, understandable, and extensible state.

The following topics will be covered:

- Reviewing the requirements for the weather application
- Crafting our initial acceptance test
- Implementing the City Search feature
- Implementing an ACL
- Implementing an Add to Favorite feature
- Fetching previous weather data when the application relaunches

Technical requirements

A GitHub repository has been created to host all the code we'll discuss in this book. For this chapter, you can find the recommended structure at `https://github.com/PacktPublishing/React-Anti-Patterns/tree/main/code/src/ch12`.

We need to complete a few more steps before we proceed. Please follow the next section to set up the necessary API keys.

Getting the OpenWeatherMap API key

To utilize OpenWeatherMap, you'll need to create an account at `https://openweathermap.org/`. Although various plans are available based on usage, the free plan suffices for our purposes. After registering, navigate to **My API keys** to find your API key, as displayed in *Figure 12.1*:

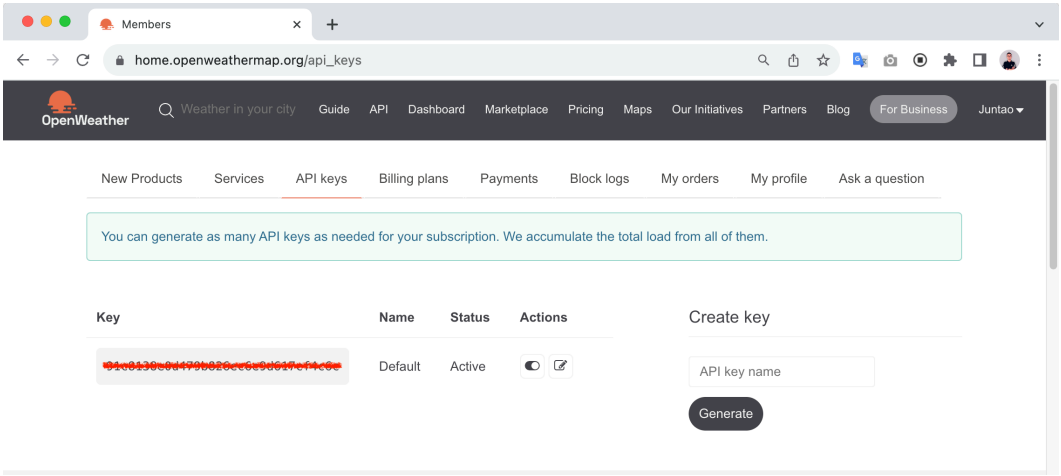

Figure 12.1: The OpenWeatherMap API key

Keep a copy of this key handy as we'll use it to make calls to the weather API so that we can fetch data.

Preparing the project's code base

If you prefer to follow along with me, you will need to install a few packages before we start. However, if you want to see the final results, they're already in the repository mentioned earlier. I recommend that you follow along to see how we evolve our application into the final state.

To kick things off, we'll create a new React app using the following commands:

```
npx create-react-app weather-app --template typescript
cd weather-app
yarn add cypress jest-fetch-mock -D
yarn install
```

These commands are used to set up a new React project with TypeScript and Cypress:

- `npx create-react-app weather-app --template typescript`: This command utilizes `npx` to run the `create-react-app` utility, which scaffolds out a new React application in a directory named `weather-app`. The `--template typescript` option specifies that this project should be configured to use TypeScript.
- `yarn add cypress jest-fetch-mock -D`: This command installs Cypress, a testing framework, as a development dependency in the project, and `jest-fetch-mock`, for mocking the `fetch` function in jest tests. The `-D` flag indicates that this is a development dependency, meaning it's not required for the production version of the application.
- `yarn install`: This command installs all the dependencies listed in the `package.json` file of the project, ensuring that all the necessary libraries and tools are available.

Finally, we can start the template application by running the following command:

```
yarn start
```

This will launch the application on port 3000. You can leave the application running on 3000 open and open another terminal window for running tests.

Reviewing the requirements for the weather application

Our envisioned weather application is designed to be a fully functional platform with the following capabilities:

- Enables users to search for cities of interest, be it their hometown, current residence, or a future travel destination
- Allows users to add cities to a favorite list, with the selection persisting locally for easy access during future visits
- Supports the addition of multiple cities to the user's list
- Ensures the site is thoroughly navigable via keyboard, facilitating ease of access for all users

The result will resemble what's illustrated in *Figure 12.2*:

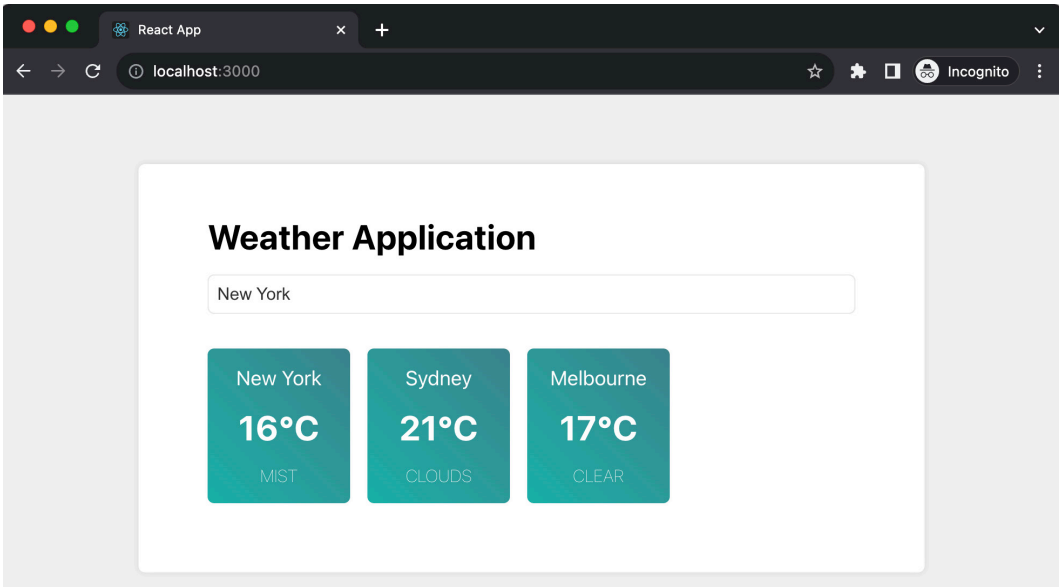

Figure 12.2: The weather application

While it's not an overly complex application, it encompasses several intriguing elements. For instance, we'll navigate through the hurdles of applying TDD in a UI application, testing Hooks, and making informed decisions on when to employ user acceptance tests versus lower-level tests.

We will commence with an initial acceptance test to ensure the application operates end-to-end, albeit it is merely verifying a single text element's appearance.

Crafting our initial acceptance test

Chapter 7 familiarized us with the notion of starting with an acceptance test – a test that's approached from an end user's standpoint, as opposed to a developer's perspective. Essentially, we aim for our test to validate aspects a user would perceive or interact with on the web page, rather than technicalities such as function calls or class initializations.

Within the folder you created in the *Technical requirements* section (that is, `weather-app`), create a Cypress test within `cypress/e2e/weather.spec.cy.ts`:

```
describe('weather application', () => {
  it('displays the application title', () => {
    cy.visit('http://localhost:3000/');
    cy.contains('Weather Application');
  });
});
```

In this code snippet, we defined a test suite named `weather application`. It uses the describe function in the Cypress testing framework. This test case comprises two main actions: navigating to the local development server at `http://localhost:3000/` using `cy.visit`, and then checking the page to ensure it contains the `Weather Application` text using `cy.contains`. If `Weather Application` is found on the page, the test will pass; if not, it will fail.

Executing the test using `npx cypress run` will, as expected, yield an error on the console due to our yet unmodified app:

```
  1) weather application
       displays the application title:
     AssertionError: Timed out retrying after 4000ms: Expected to find
      content: 'Weather Application' but never did.
      at Context.eval (webpack://tdd-weather/./cypress/e2e/weather.
       spec.cy.ts:4:7)
```

This error reveals that it was anticipating the `Weather Application` text but didn't locate it within the default 4-second timeout specified by Cypress. To rectify this, we'll need to tweak `App.tsx` so that it includes this text.

After clearing the current content in `App.tsx` (generated by `create-react-app`), we'll insert a simple `h1` tag to display the text:

```
import React from 'react';

function App() {
  return (
    <div className="App">
      <h1>Weather Application</h1>
    </div>
  );
}
```

This code defines a functional component named `App` in React, which renders a `div` element containing an `h1` element with the `Weather Application` text. With this heading defined, our Cypress test will pass.

Now, let's move on to the first meaningful feature – allowing users to search by a city name.

Implementing a City Search feature

Let's commence the development of our first feature – City Search. Users will be able to enter a city name into a search box, which triggers a request to a remote server. Upon receiving the data, we'll render it into a list for user selection. Throughout this chapter, we'll utilize the OpenWeatherMap API for city searches as well as retrieving weather information.

Introducing the OpenWeatherMap API

OpenWeatherMap is a service that offers global weather data via an API, allowing users to access current, forecasted, and historical weather data for any location worldwide. It's a popular choice among developers for embedding real-time weather updates into apps and websites.

We're going to use two APIs in our weather application – one for searching the city by name and another for getting the actual real-time weather. To use the API, you'll need the API key you received when following the instructions in the *Technical requirements* section.

You can try sending the request to OpenWeatherMap either in your browser or with a command-line tool such as `curl` or `http` (from `https://httpie.io/`):

```
http https://api.openweathermap.org/geo/1.0/direct?q="Melbourne"&limit
=5&appid=<your-app-key>
```

This line utilizes the `http` command to send an HTTP request to the OpenWeatherMap API, specifically to its geocoding endpoint (`geo/1.0/direct`), looking up cities named `Melbourne`, with a limit of 5 results. The `appid` (as specified in the preceding URL) parameter is where you need to insert *your* OpenWeatherMap API key to authenticate the request.

So, the command fetches basic geocoding information about cities named Melbourne, which can later be used to get weather data for those locations. You will get the result in JSON format, like so:

```
[
    {
        "country": "AU",
        "lat": -37.8142176,
        "local_names": {},
        "lon": 144.9631608,
        "name": "Melbourne",
        "state": "Victoria"
    },
    {
        "country": "US",
        "lat": 28.106471,
        "local_names": {
        },
        "lon": -80.6371513,
        "name": "Melbourne",
        "state": "Florida"
    }
]
```

Be aware that the OpenWeatherMap free plan comes with a rate limit, restricting us to a maximum of 60 requests per minute and 1,000,000 requests per month. While these limits may seem high, the number of requests can accumulate quickly during development through testing and debugging. To conserve our request allowance, we'll avoid making requests to the real server and instead stub these requests, returning predefined values. For a refresher on stubbing, refer to *Chapter 5*.

Let's save the resultant data into a text file named `search-results.json`, located under `cypress/fixtures/search-result.json`.

Stubbing the search results

With the file in place, we can write a test for the **Search city by name** feature. As we don't send the real request to OpenWeatherMap, we'll intercept network requests that are sent to the API and return the file content we created previously – that is, `fixtures/search-result.json`:

```
import searchResults from '../fixtures/search-result.json';

describe('weather application', () => {
  //...

  it('searches for a city', () => {
    cy.intercept("GET", "https://api.openweathermap.org/geo/1.0/
     direct?q=*", {
      statusCode: 200,
      body: searchResults,
    });

    cy.visit('http://localhost:3000/');

    cy.get('[data-testid="search-input"]').type('Melbourne');
    cy.get('[data-testid="search-input"]').type('{enter}');

    cy.get('[data-testid="search-results"] .search-result')
      .should('have.length', 5);
  });
});
```

Here, we created a test case called `'searches for a city'`. The test case does the following:

1. First, it sets up an interception for a `GET` request that goes to the OpenWeatherMap API for city searches. Whenever a request matching the criteria is made, it responds with a 200 status code and the body content from a predefined `searchResults` file, essentially mocking the API response.

2. Then, it navigates to the application running on `http://localhost:3000/`.
3. Next, it simulates a user typing `Melbourne` into an input field with a `data-testid` value of `search-input` and pressing the *Enter* key.
4. Lastly, it checks that a container with a `data-testid` value of `"search-results"` contains exactly five elements with a class of `search-result`, which are the search results that are returned from the stubbed API request. This verifies that the application correctly displays the search results.

We're in the red step in TDD (the first step, which indicates that a test is failing), so let's go to our application code, `App.tsx`, to fix the test:

```
function App() {
  const [query, setQuery] = useState<string>("");
  const [searchResults, setSearchResults] = useState<any[]>([]);

  const handleKeyDown = (e: KeyboardEvent<HTMLInputElement>) => {
    if (e.key === "Enter") {
      fetchCities();
    }
  };

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setQuery(e.target.value);
  };

  const fetchCities = () => {
    fetch(
      `https://api.openweathermap.org/geo/1.0/direct?q=${query}&limit=
       5&appid=<app-key>`
    )
      .then((r) => r.json())
      .then((cities) => {
        setSearchResults(
          cities.map((city: any) => ({
            name: city.name,
          }))
        );
      });
  };

  return (
    <div className="app">
```

```
      <h1>Weather Application</h1>

      <div className="search-bar">
        <input
          type="text"
          data-testid="search-input"
          onKeyDown={handleKeyDown}
          onChange={handleChange}
          placeholder="Enter city name (e.g. Melbourne, New York)"
        />
      </div>

      <div className="search-results-popup">
        {searchResults.length > 0 && (
          <ul data-testid="search-results">
            {searchResults.map((city, index) => (
              <li key={index} className="search-result">
                {city.name}
              </li>
            ))}
          </ul>
        )}
      </div>
    </div>
  );
}
```

The `App` function in this code sets up a simple weather application using React. It initializes the `query` and `searchResults` state variables to handle user input and display search results, respectively. The `handleKeyDown` and `handleChange` event handlers are set up to update the search query and trigger a city search when the user presses *Enter*. The `fetchCities` function sends a request to the OpenWeatherMap API, processes the response to extract the city names, and updates `searchResults`.

In the TSX part, an input field is provided for the user to type a city name, and a list displays the search results whenever there are any available.

The tests are now passing with these changes, and we can launch our browser to access the application. As depicted in *Figure 12.3*, our implementation now displays a drop-down list of cities upon conducting a search:

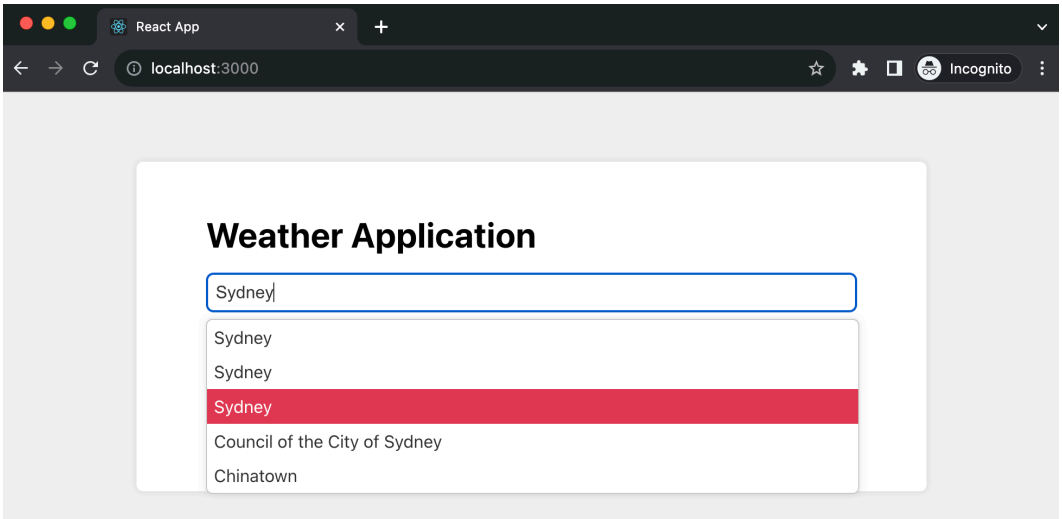

Figure 12.3: The search result dropdown

> **Note**
>
> I've incorporated some CSS to enhance the visual appeal. However, to maintain focus on the core content, the CSS has not been included here. For a complete understanding, you can refer to the repository mentioned in the *Technical requirements* section to see the full implementation.

Enhancing the search result list

As we utilize city names for search queries, it's common to encounter multiple matches in the search results. To refine this, we could enrich each item with additional details such as the state name, country name, or even coordinates to make the results more distinctive.

Following the TDD approach, we'll start with a test. Although a Cypress test could be crafted, detailing such aspects is better suited for lower-level tests such as unit tests. Cypress tests are end-to-end, encompassing all the parts – pages, network (even with interceptors) – and from its standpoint, it perceives no components, just HTML, CSS, and JavaScript. This makes them more costly to run, extending the feedback loop compared to lower-level tests, which generally operate in an in-memory browser and focus on isolated areas.

For this enhancement, we'll employ Jest tests, which are more lightweight, faster, and offer specificity in test case writing.

In the following snippet, we aim to test if an item displays the city name:

```
it("shows a city name", () => {
  render(<SearchResultItem item={{ city: "Melbourne" }} />);
  expect(screen.getByText("Melbourne")).toBeInTheDocument();
});
```

Here, we invoke the `render` method from the React Testing Library on a `SearchResultItem` component, passing a prop item with a `city` field. Then, we assert that the `Melbourne` text is present in the document.

At this point, we lack a `SearchResultItem` component ready for testing. However, a slight refactoring can help us extract one. Let's create a `SearchResultItem.tsx` file and define the component as follows:

```
export const SearchResultItem = ({ item }: { item: { city: string } })
=> {
  return <li className="search-result">{item.city}</li>;
};
```

Now, integrate the component in `App.tsx`:

```
function App() {
  //...
  <div className="search-results-popup">
    {searchResults.length > 0 && (
      <ul data-testid="search-results" className="search-results">
        {searchResults.map((city, index) => (
          <SearchResultItem key={index} item={{ city }} />
        ))}
      </ul>
    )}
  </div>
  //...
}
```

In this part of `App.tsx`, we map through `searchResults`, rendering `SearchResultItem` for each city, and passing the city data as a prop.

Now, let's extend our tests to check for the city name, state, and country:

```
it("shows a city name, the state, and the country", () => {
  render(
    <SearchResultItem
      item={{ city: "Melbourne", state: "Victoria", country:
       "Australia" }}
    />
  );
  expect(screen.getByText("Melbourne")).toBeInTheDocument();
  expect(screen.getByText("Victoria")).toBeInTheDocument();
  expect(screen.getByText("Australia")).toBeInTheDocument();
});
```

Next, to accommodate these new fields, we'll tweak the type definition of `SearchResultItem` and render the passed props:

```
type SearchResultItemProps = {
  city: string;
  state: string;
  country: string;
};

export const SearchResultItem = ({ item }: { item:
SearchResultItemProps }) => {
  return (
    <li className="search-result">
      <span>{item.city}</span>
      <span>{item.state}</span>
      <span>{item.country}</span>
    </li>
  );
};
```

Here, we define a `SearchResultItemProps` type to specify the shape of the item prop, ensuring it contains the `city`, `state`, and `country` fields. Our `SearchResultItem` component then renders these fields within a list item, each in a separate `span` element.

As you can see, now, the list item provides more details to help users distinguish between the results:

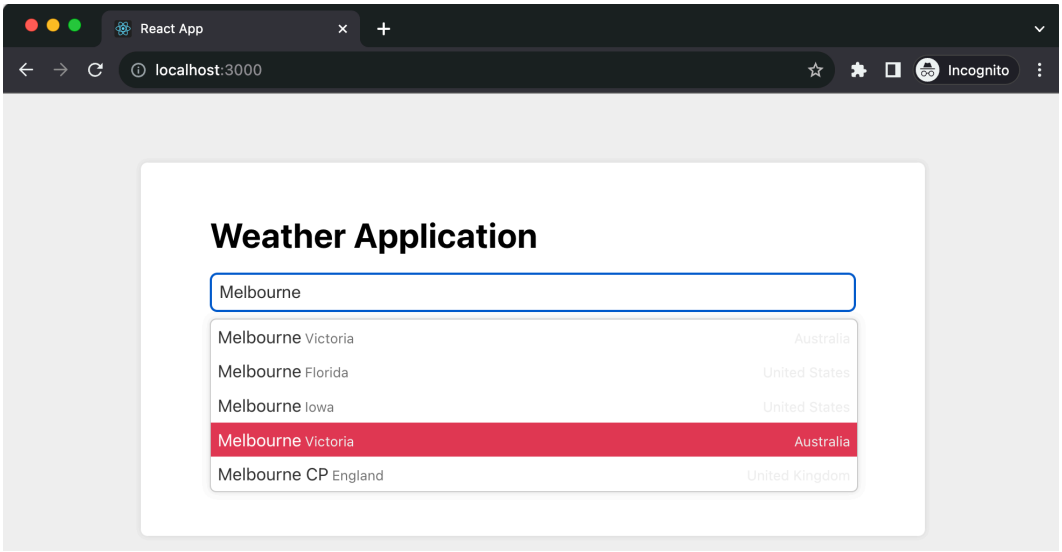

Figure 12.4: Enhanced city drop-down list

Before we proceed to the next major feature, let's tackle some housekeeping tasks. While we've been focused on delivering the feature, we haven't given much attention to code quality so far, so let's do that.

Implementing an ACL

Within our application, the `SearchResultItem` component serves its purpose well. However, the challenge arises from the discrepancy between the data shape we require and the data shape we receive from the remote server.

Consider the server's response:

```
[
    {
    "country": "US",
    "lat": 28.106471,
    "local_names": {
      "en": "Melbourne",
      "ja": "メルボーン",
      "ru": "Мельбурн",
      "uk": "Мелборн"
    },
    "lon": -80.6371513,
    "name": "Melbourne",
    "state": "Florida"
  }
]
```

The server's response includes many elements we don't need. Moreover, we want to shield our `SearchResultItem` component from any future changes to the data shape from the server.

As we discussed in *Chapter 8*, we can employ an ACL to address this issue. Using this, we aim to map the city name and state directly, but for the country, we want to display its full name to avoid any ambiguity in the UI.

To do this, first, we must define a `RemoteSearchResultItem` type to represent the remote data shape:

```
interface RemoteSearchResultItem {
  city: string;
  state: string;
  country: string;

  lon: number;
  lat: number;

  local_names: {
```

```
    [key: string]: string
  }
}
```

Next, we must change the `SearchResultItemProps` type to a class, enabling its initialization within the TypeScript code:

```
const countryMap = {
  "AU": "Australia",
  "US": "United States",
  "GB": "United Kingdom"
  //...
}

class SearchResultItemType {
  private readonly _city: string;
  private readonly _state: string;
  private readonly _country: string;

  constructor(item: RemoteSearchResultItem) {
    this._city = item.city;
    this._state = item.state;
    this._country = item.country
  }

  get city() {
    return this._city
  }

  get state() {
    return this._state
  }

  get country() {
    return countryMap[this._country] || this._country;
  }
}
```

This segment of code defines a class, `SearchResultItemType`, that accepts a `RemoteSearchResultItem` object in its `constructor` and initializes its properties accordingly. It also provides getter methods to access these properties, with a special handler for the country property to the map country code to its full name.

Now, our `SearchResultItem` component can utilize this newly defined class:

```
import React from "react";
import { SearchResultItemType } from "./models/SearchResultItemType";

export const SearchResultItem = ({ item }: { item:
SearchResultItemType }) => {
  return (
    <li className="search-result">
      <span>{item.city}</span>
      <span>{item.state}</span>
      <span>{item.country}</span>
    </li>
  );
};
```

Note how we use the `item.city` and `item.state` getter functions, just like a regular JavaScript object.

Then, in our Jest tests, we can verify the transformation logic straightforwardly, as seen here:

```
it("converts the remote type to local", () => {
  const remote = {
    country: "US",
    lat: 28.106471,
    local_names: {
      en: "Melbourne",
      ja: "メルボーン",
      ru: "Мельбурн",
      uk: "Мелборн",
    },
    lon: -80.6371513,
    name: "Melbourne",
    state: "Florida",
  };

  const model = new SearchResultItemType(remote);

  expect(model.city).toEqual('Melbourne');
  expect(model.state).toEqual('Florida');
  expect(model.country).toEqual('United States');
});
```

In this test, we create an instance of `SearchResultItemType` using a mock `RemoteSearchResultItem` object and verify that the transformation logic works as intended – the fields are correctly mapped and the country has its full name too.

Once the tests confirm the expected behavior, we can apply this new class within our application code, as follows:

```
const fetchCities = () => {
  fetch(
    `https://api.openweathermap.org/geo/1.0/direct?q=${query}&limit=5&
     appid=<api-key>`
  )
    .then((r) => r.json())
    .then((cities) => {
      setSearchResults(
        cities.map(
          (item: RemoteSearchResultItem) => new
           SearchResultItemType(item)
        )
      );
    });
};
```

This function fetches city data from the remote server, transforms the received data into instances of `SearchResultItemType`, and then updates the `searchResults` state.

With enriched details in the dropdown, users can identify their desired city. Having achieved this, we can proceed to allow users to add cities to their favorite list, paving the way to display weather information for these selected cities.

Given our Cypress feature tests, there's a safeguard against inadvertently breaking functionality. Additionally, with the newly incorporated unit tests, any discrepancies between remote and local data shapes will be automatically detected. We are now well-positioned to embark on developing the next feature.

Implementing an Add to Favorite feature

Let's investigate implementing our next feature: `'adds city to favorite list'`. Because this is a feature that is critical in the weather application, we want to make sure users can see the city being added and that the dropdown is closed.

First, we'll start with another Cypress test:

```
it('adds city to favorite list', () => {
  cy.intercept("GET", "https://api.openweathermap.org/geo/1.0/
direct?q=*", {
    statusCode: 200,
    body: searchResults,
  });
```

```
  cy.visit('http://localhost:3000/');

  cy.get('[data-testid="search-input"]').type('Melbourne');
  cy.get('[data-testid="search-input"]').type('{enter}');

  cy.get('[data-testid="search-results"] .search-result')
    .first()
    .click();

  cy.get('[data-testid="favorite-cities"] .city')
    .should('have.length', 1);

  cy.get('[data-testid="favorite-cities"]
   .city:contains("Melbourne")').should('exist');
  cy.get('[data-testid="favorite-cities"] .city:contains("20°C")').
   should('exist');
})
```

In the test, we set up an intercept to mock a `GET` request to the OpenWeatherMap API and then visit the app running on localhost. From here, it simulates typing `Melbourne` into a search input and hitting *Enter*. After that, it clicks on the first search result and checks if the favorite cities list now contains one city. Finally, it verifies that the favorite cities list contains a city element with `Melbourne` and `20°C`.

Please note that there are a few things that need a bit more explanation in the last two lines:

- `cy.get(selector)`: This Cypress command is used to query DOM elements on a page. It's similar to `document.querySelector`. Here, it's being used to select elements with specific text content inside a particular part of the DOM. Cypress supports not only the basic CSS selectors, such as class and ID selectors, but also advanced selectors such as `.city:contains("Melbourne")`, so we can use a selector for a more specific selection.
- `.city:contains(text)`: This is a jQuery-style selector that Cypress supports. It allows you to select an element containing specific text. In this case, it's being used to find elements within `[data-testid="favorite-cities"]` that have a class of `city` and contain `Melbourne` or `20°C`.
- `.should('exist')`: This is a Cypress command that asserts that the selected elements should exist in the DOM. If the element does not exist, the test will fail.

Now, to get the weather for the city, we need another API endpoint:

```
http https://api.openweathermap.org/data/2.5/weather?lat=-
37.8142176&lon=144.9631608&appid=<api-key>&units=metric
```

The API requires two parameters: the latitude and longitude.

Then, it returns the current weather in this format:

```
{
  //...
  "main": {
      "feels_like": 20.75,
      "humidity": 56,
      "pressure": 1009,
      "temp": 20.00,
      "temp_max": 23.46,
      "temp_min": 18.71
  },
  "name": "Melbourne",
  "timezone": 39600,
  "visibility": 10000,
  "weather": [
      {
          "description": "clear sky",
          "icon": "01d",
          "id": 800,
          "main": "Clear"
      }
  ],
  //...
}
```

There are many fields in the response but we only need some of them for now. We can intercept the request and serve the response in the Cypress test, just like we did for the city search API:

```
cy.intercept('GET', 'https://api.openweathermap.org/data/2.5/
weather*', {
  fixture: 'melbourne.json'
}).as('getWeather')
```

Transitioning to the implementation, we'll weave an `onClick` event handler into `SearchResultItem`; upon clicking an item, an API call will be triggered, followed by the addition of a city to a list designated for rendering:

```
export const SearchResultItem = ({
  item,
  onItemClick,
}: {
  item: SearchResultItemType;
```

```
  onItemClick: (item: SearchResultItemType) => void;
}) => {
  return (
    <li className="search-result" onClick={() => onItemClick(item)}>
    { /* JSX for rendering the item details */ }
    </li>
  );
};
```

Now, let's dive into the application code to interlace the data-fetching logic:

```
const onItemClick = (item: SearchResultItemType) => {
  fetch(
    `http https://api.openweathermap.org/data/2.5/weather?lat=${item.
latitude}&lon=${item.longitude}&appid=<api-key>&units=metric`
  )
    .then((r) => r.json())
    .then((cityWeather) => {
      setCity({
        name: cityWeather.name,
        degree: cityWeather.main.temp,
      });
    });
};
```

The `onItemClick` function is triggered upon clicking a city item. It makes a network request to the OpenWeatherMap API using the item's latitude and longitude, fetching the current weather data for the selected city. Then, it parses the response to JSON, extracts the city name and temperature from the parsed data, and updates the city state using the `setCity` function, which will cause the component to re-render and display the selected city's name and current temperature.

Note the `SearchResultItemType` parameter in the previous snippet. We'll need to extend this type so that it encompasses latitude and longitude. We can achieve this by revisiting the ACL layer in the `SearchResultItemType` class:

```
class SearchResultItemType {
  //... the city, state, country as before
  private readonly _lat: number;
  private readonly _long: number;

  constructor(item: RemoteSearchResultItem) {
    //... the city, state, country as before
    this._lat = item.lat;
    this._long = item.lon;
```

```
  }

  get latitude() {
    return this._lat;
  }

  get longitude() {
    return this._long;
  }
}
```

With that, we have extended `SearchResultItemType` with two new fields, `latitude` and `longitude`, which will be used in the API query.

Finally, upon successfully retrieving the city data, it's time for rendering:

```
function App() {
  const [city, setCity] = useState(undefined);

  const onItemClick = (item: SearchResultItemType) => {
    //...
  }

  return(
      <div className="search-results-popup">
        {searchResults.length > 0 && (
          <ul data-testid="search-results">
            {searchResults.map((item, index) => (
              <SearchResultItem
                key={index}
                item={item}
                onItemClick={onItemClick}
              />
            ))}
          </ul>
        )}
      </div>

      <div data-testid="favorite-cities">
        {city && (
          <div className="city">
            <span>{city.name}</span>
            <span>{city.degree}°C</span>
          </div>
        )}
```

```
        </div>
    );
}
```

In this block of code, the `onItemClick` function is assigned as the `onClick` event handler for each `SearchResultItem`. When a city is clicked, the `onItemClick` function is invoked, triggering a fetch request to retrieve the weather data for the selected city. Once the data is obtained, the `setCity` function updates the city state, which then triggers a re-render, displaying the selected city in the **Favorite Cities** section.

Now that all the tests have passed, it signifies that our implementation aligns with the expectations up to this point. However, before we proceed to the next enhancement, it's essential to undertake some refactoring to ensure our code base remains robust and easily maintainable.

Modeling the weather

Just as we modeled the city search result, it's necessary to model the weather for similar reasons – to isolate our implementation from the remote data shape, and to centralize data shape conversion, fallback logic, and so on.

Several areas require refinement. We must do the following:

- Ensure all the relevant data is typed
- Create a data model for the weather to centralize all formatting logic
- Utilize fallback values within the data model when certain data is unavailable

Let's begin with the remote data type, `RemoteCityWeather`:

```
interface RemoteCityWeather {
  name: string;
  main: {
    temp: number;
    humidity: number;
  };
  weather: [{
    main: string;
    description: string;
  }];
  wind: {
    deg: number;
    speed: number;
  };
}

export type { RemoteCityWeather };
```

Here, we defined a type called `RemoteCityWeather` to reflect the remote data shape (and have also filtered out a few fields that we don't use).

Then, we must define a new type called `CityWeather` for our UI to use CityWeather:

```
import { RemoteCityWeather } from "./RemoteCityWeather";

export class CityWeather {
  private readonly _name: string;
  private readonly _main: string;
  private readonly _temp: number;

  constructor(weather: RemoteCityWeather) {
    this._name = weather.name;
    this._temp = weather.main.temp;
    this._main = weather.weather[0].main;
  }

  get name() {
    return this._name;
  }

  get degree() {
    return Math.ceil(this._temp);
  }

  get temperature() {
    if (this._temp == null) {
      return "-/-";
    }
    return `${Math.ceil(this._temp)}°C`;
  }

  get main() {
    return this._main.toLowerCase();
  }
}
```

This code defines a `CityWeather` class to model city weather data. It takes a `RemoteCityWeather` object as a constructor argument and initializes private fields from it – that is, `_name`, `_temp`, and `_main`. The class provides getter methods to access a city's name, rounded temperature in degrees, formatted temperature string, and the weather description in lowercase.

For the temperature's `getter` method, if `_temp` is null or undefined, it returns a string, `-/-`. Otherwise, it proceeds to calculate the ceiling (rounding up to the nearest whole number) of `_temp`

using `Math.ceil(this._temp)`, appends a degree symbol to it, and returns this formatted string. This way, the method provides a fallback value of `-/-` when `_temp` is not set while formatting the temperature value when `_temp` is set.

Now, in `App`, we can use the calculated logic:

```
const onItemClick = (item: SearchResultItemType) => {
  fetch(
    `https://api.openweathermap.org/data/2.5/weather?lat=${item.
latitude}&lon=${item.longitude}&appid=<api-key>&units=metric`
  )
    .then((r) => r.json())
    .then((cityWeather: RemoteCityWeather) => {
      setCity(new CityWeather(cityWeather));
      setDropdownOpen(false);
    });
};
```

The `onItemClick` function is triggered when a city item is clicked. It makes a `fetch` request to the OpenWeatherMap API using the latitude and longitude of the clicked city item. Upon receiving the response, it converts the response into JSON, then creates a new `CityWeather` instance with the received data, and updates the city state using `setCity`.

Additionally, it closes the drop-down menu by setting the `setDropdownOpen` state to `false`. If we don't close it off, the Cypress test will not be able to "see" the underlying weather information, which will cause the test to fail, as shown in the following screenshot:

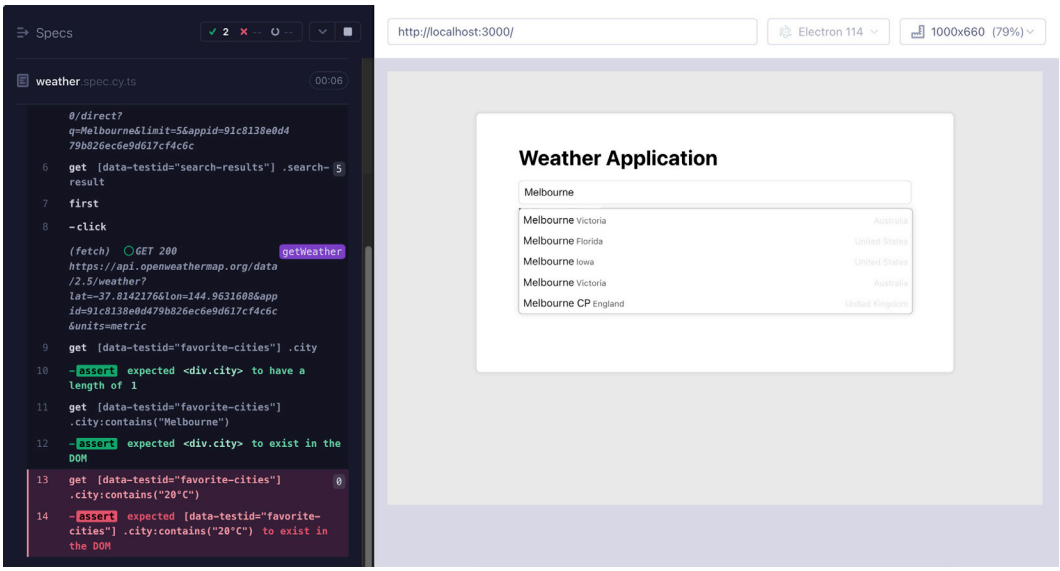

Figure 12.5: The Cypress test failed because the weather is covered

Then, we must render the selected city details correspondingly:

```
<div data-testid="favorite-cities">
  {city && (
    <div className="city">
      <span>{city.name}</span>
      <span>{city.temperature}</span>
    </div>
  )}
</div>
```

For rendering the city part, if the city state is defined (that is, a city has been selected), it displays a `div` element with a class name of `city`. Inside this `div` element, we can see the city's name and temperature using the `city.name` and `city.temperature` properties, respectively.

Now, with some additional styling, our application looks like this:

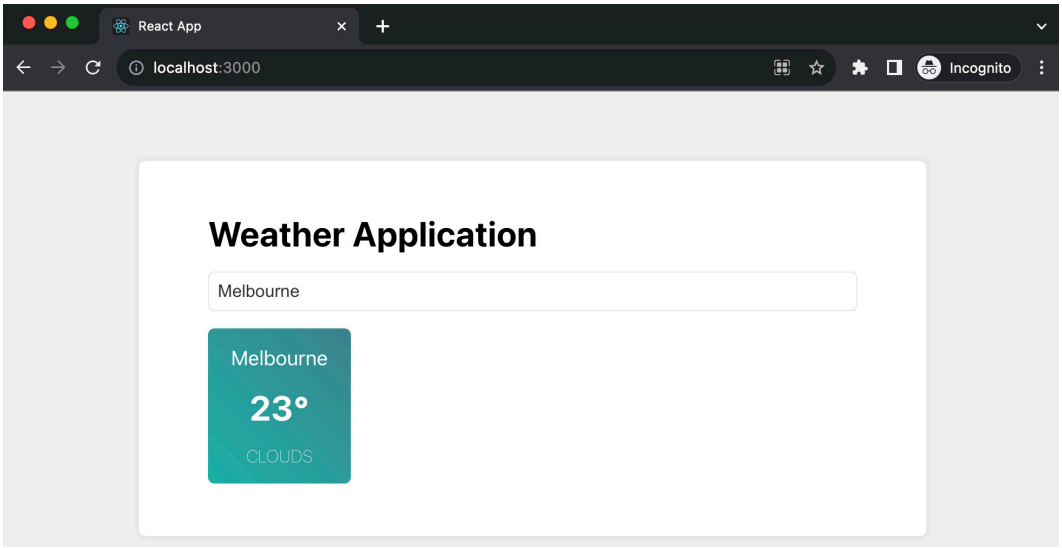

Figure 12.6: Adding a city to the favorite list

We've done an excellent job with data modeling and have set up a solid ACL to bolster our UI's robustness and ease of maintenance. Nonetheless, upon examining the root `App` component, we'll undoubtedly find areas that require improvement.

Refactoring the current implementation

Our current `App` component has grown too lengthy for easy readability and feature additions, indicating a need for some refactoring to tidy things up. It doesn't adhere well to the single responsibility principle

as it takes on multiple responsibilities: handling network requests for city search and weather queries, managing the state of the dropdown's open and closed status, and several event handlers.

One approach to realigning with better design principles is to break the UI into smaller components. Another is utilizing custom Hooks for state management. Given that a significant portion of the logic here revolves around managing the state for the city search dropdown, it's sensible to start by isolating this part.

Let's start by extracting all city search-related logic into a custom Hook:

```
const useSearchCity = () => {
  const [query, setQuery] = useState<string>("");
  const [searchResults, setSearchResults] =
   useState<SearchResultItemType[]>(
     []
  );
  const [isDropdownOpen, setDropdownOpen] = useState<boolean>(false);

  const fetchCities = () => {
    fetch(
      `https://api.openweathermap.org/geo/1.0/direct?q=${query}&limit=
       5&appid=<api-key>`
    )
      .then((r) => r.json())
      .then((cities) => {
        setSearchResults(
          cities.map(
            (item: RemoteSearchResultItem) => new
             SearchResultItemType(item)
          )
        );
        openDropdownList();
      });
  };

  const openDropdownList = () => setDropdownOpen(true);
  const closeDropdownList = () => setDropdownOpen(false);

  return {
    fetchCities,
    setQuery,
    searchResults,
    isDropdownOpen,
    openDropdownList,
    closeDropdownList,
```

```
  };
};

export { useSearchCity };
```

The `useSearchCity` Hook manages the city search functionality. It initializes states for query, search results, and dropdown open status using `useState`. The `fetchCities` function triggers a network request to fetch cities based on the query, processes the response to create `SearchResultItemType` instances, updates the search results state, and opens the drop-down list. Two functions, `openDropdownList` and `closeDropdownList`, are defined to toggle the dropdown's open status. The Hook returns an object containing these functionalities, which can be used by components that import and invoke `useSearchCity`.

Next, we extract a component, `SearchCityInput`, to handle all the search input-related work: to handle the *Enter* key for performing the search, open the drop-down list, and handle users clicking on each item:

```
export const SearchCityInput = ({
  onItemClick,
}: {
  onItemClick: (item: SearchResultItemType) => void;
}) => {
  const {
    fetchCities,
    setQuery,
    isDropdownOpen,
    closeDropdownList,
    searchResults,
  } = useSearchCity();

  const handleKeyDown = (e: KeyboardEvent<HTMLInputElement>) => {
    if (e.key === "Enter") {
      fetchCities();
    }
  };

  const handleChange = (e: ChangeEvent<HTMLInputElement>) =>
    setQuery(e.target.value);

  const handleItemClick = (item: SearchResultItemType) => {
    onItemClick(item);
    closeDropdownList();
  };

  return (
```

```
    <>
      <div className="search-bar">
        <input
          type="text"
          data-testid="search-input"
          onKeyDown={handleKeyDown}
          onChange={handleChange}
          placeholder="Enter city name (e.g. Melbourne, New York)"
        />
      </div>

      {isDropdownOpen && (
        //... render the dropdown
      )}
    </>
  );
};
```

The `SearchCityInput` component is responsible for rendering and managing the user input for searching cities, utilizing the `useSearchCity` Hook to access search-related functionalities.

The `handleKeyDown` and `handleChange` functions are defined to handle user interactions, triggering a search upon pressing *Enter* and updating the query on input change, respectively. Then, the `handleItemClick` function is defined to handle the action when a search result item is clicked, which triggers the `onItemClick` prop function and closes the drop-down list.

In the `render` method, an input field is provided for the user to type the search query, and the drop-down list is conditionally rendered based on the `isDropdownOpen` state. If the dropdown is open and there are search results, a list of `SearchResultItem` components is rendered, each being passed the current item data and the `handleItemClick` function.

For all the logic of the weather for a city, we can extract another Hook, `useCityWeather`:

```
const useFetchCityWeather = () => {
  const [cityWeather, setCityWeather] = useState<CityWeather |
   undefined>(undefined);

  const fetchCityWeather = (item: SearchResultItemType) => {
    fetch(
      `https://api.openweathermap.org/data/2.5/weather?lat=${item.
latitude}&lon=${item.longitude}&appid=<api-key>&units=metric`
    )
      .then((r) => r.json())
      .then((cityWeather: RemoteCityWeather) => {
        setCityWeather(new CityWeather(cityWeather));
```

```
      });
  };

  return {
    cityWeather,
    fetchCityWeather,
  };
};
```

The `useFetchCityWeather` custom Hook is designed to manage the fetching and storing of weather data for a specified city. It maintains a state, `cityWeather`, to hold the weather data. The Hook provides a function, `fetchCityWeather`, which takes a `SearchResultItemType` object as an argument to get the `latitude` and `longitude` values for the API call.

Upon receiving the response, it processes the JSON data, creates a new `CityWeather` object from the `RemoteCityWeather` data, and updates the `cityWeather` state with it. The Hook returns the `cityWeather` state and the `fetchCityWeather` function for use in other components (`Weather`, for example).

We could extract a `Weather` component to accept `cityWeather` and render it:

```
const Weather = ({ cityWeather }: { cityWeather: CityWeather |
undefined }) => {
  if (cityWeather) {
    return (
      <div className="city">
        <span>{cityWeather.name}</span>
        <span>{cityWeather.degree}°C</span>
      </div>
    );
  }

  return null;
};
```

The `Weather` component accepts a prop, `cityWeather`. If `cityWeather` is defined, the component renders a `div` element with the `city` class name, displaying the city's name and temperature in degrees Celsius. If `cityWeather` is undefined, it returns `null`.

With the extracted Hook and component, our `App.tsx` is simplified into something like this:

```
function App() {
  const { cityWeather, fetchCityWeather } = useFetchCityWeather();

  const onItemClick = (item: SearchResultItemType) =>
```

```
    fetchCityWeather(item);

  return (
    <div className="app">
      <h1>Weather Application</h1>

      <SearchCityInput onItemClick={onItemClick} />

      <div data-testid="favorite-cities">
        <Weather cityWeather={cityWeather} />
      </div>
    </div>
  );
}
```

In the `App` function, we're utilizing the `useFetchCityWeather` custom Hook to obtain `cityWeather` and `fetchCityWeather` values. The `onItemClick` function is defined to call `fetchCityWeather` with an item of the `SearchResultItemType` type. In the rendering part, we can now simply use the component and functions we extracted.

If we open the project folder to examine the current folder structure, we will see that we have different elements defined in distinct modules:

```
src
├── App.tsx
├── index.tsx
├── models
│   ├── CityWeather.ts
│   ├── RemoteCityWeather.ts
│   ├── RemoteSearchResultItem.ts
│   ├── SearchResultItemType.test.ts
│   └── SearchResultItemType.ts
├── search
│   ├── SearchCityInput.tsx
│   ├── SearchResultItem.test.tsx
│   ├── SearchResultItem.tsx
│   └── useSearchCity.ts
└── weather
    ├── Weather.tsx
    ├── useFetchCityWeather.test.ts
    ├── useFetchCityWeather.ts
    └── weather.css
```

Now, after all that work, each module possesses a clearer boundary and defined responsibility. If you wish to delve into the search functionality, `SearchCityInput` is your starting point. For insights on the actual search execution, the `useSearchCity` Hook is where you'd look. Each level maintains its own abstraction and distinct responsibility, simplifying code comprehension and maintenance significantly.

Since our code is in a great state and is ready for us to add more features on top of it, we can look into enhancing the current feature.

Enabling multiple cities in the favorite list

Now, let's examine a specific example to illustrate the ease with which we can expand the existing code with a simple feature upgrade. A user may have several cities they're interested in, but at present, we can only display one city.

To allow multiple cities in the favorite list, which component should we modify to implement this change? Correct – the `useFetchCityWeather` Hook. To enable it so that it can manage a list of cities, we'll need to display this list within `App`. There's no need to delve into the city search-related files, indicating that this structure has halved the time we would spend sifting through files.

As we're performing TDD, let's write a test for the Hook first:

```
const weatherAPIResponse = JSON.stringify({
  main: {
    temp: 20.0,
  },
  name: "Melbourne",
  weather: [
    {
      description: "clear sky",
      main: "Clear",
    },
  ],
});

const searchResultItem = new SearchResultItemType({
  country: "AU",
  lat: -37.8141705,
  lon: 144.9655616,
  name: "Melbourne",
  state: "Victoria",
});
```

Firstly, let's define some data we want to test against. We can initialize data with `weatherAPIResponse`, which contains a mock response from a weather API in JSON string format, and `searchResultItem`, which holds an instance of `SearchResultItemType` with location details for Melbourne, Australia.

For the actual test case, we'll need to use `fetchMock` from `jest-fetch-mock`; we installed it in the *Technical requirements* section:

```
describe("fetchCityWeather function", () => {
  beforeEach(() => {
    fetchMock.resetMocks();
  });

  it("returns a list of cities", async () => {
    fetchMock.mockResponseOnce(weatherAPIResponse);

    const { result } = renderHook(() => useFetchCityWeather());

    await act(async () => {
      await result.current.fetchCityWeather(searchResultItem);
    });

    await waitFor(() => {
      expect(result.current.cities.length).toEqual(1);
      expect(result.current.cities[0].name).toEqual("Melbourne");
    });
  });
});
```

The preceding code sets up a test suite for the `fetchCityWeather` function. Before each test, it resets any mocked fetch calls. The test case aims to verify that the function returns a list of cities. It mocks an API response using `fetchMock.mockResponseOnce`, then invokes the `useFetchCityWeather` custom Hook. The `fetchCityWeather` function is called within an `act` block to handle state updates. Finally, the test asserts that the returned list of cities contains one city, **Melbourne**.

This setup helps test the `fetchCityWeather` function in isolation, ensuring it behaves as expected when provided with a specific input and when receiving a specific API response.

Correspondingly, we need to update the `useFetchCityWeather` Hook to enable multiple items:

```
const useFetchCityWeather = () => {
  const [cities, setCities] = useState<CityWeather[]>([]);

  const fetchCityWeather = (item: SearchResultItemType) => {
    //... fetch
      .then((cityWeather: RemoteCityWeather) => {
```

```
        setCities([new CityWeather(cityWeather), ...cities]);
      });
  };

  return {
    cities,
    fetchCityWeather,
  };
};
```

The `useFetchCityWeather` Hook now maintains a state of an array of `CityWeather` objects called `cities`. We still send requests to the `OpenWeatherMap` API and then we make sure the new item is inserted at the start of the list when it's fetched. It returns an object containing the `cities` array to the calling place.

Finally, in `App`, we can iterate over `cities` to generate the `Weather` component for each one:

```
function App() {
  //...
  const { cities, fetchCityWeather } = useFetchCityWeather();

  return (
    <div className="app">
      {/* other jsx */}
      <div data-testid="favorite-cities">
        {cities.map((city) => (
          <Weather key={city.name} cityWeather={city} />
        ))}
      </div>
    </div>
  );
}
```

The `cities` array is mapped over, and for each `CityWeather` object in the array, a `Weather` component is rendered. The key prop for each `Weather` component is set to the city's name, and the `cityWeather` prop is set to the `CityWeather` object itself, which will display the weather information for each city in the list.

Now, we will be able to see something like this in the UI:

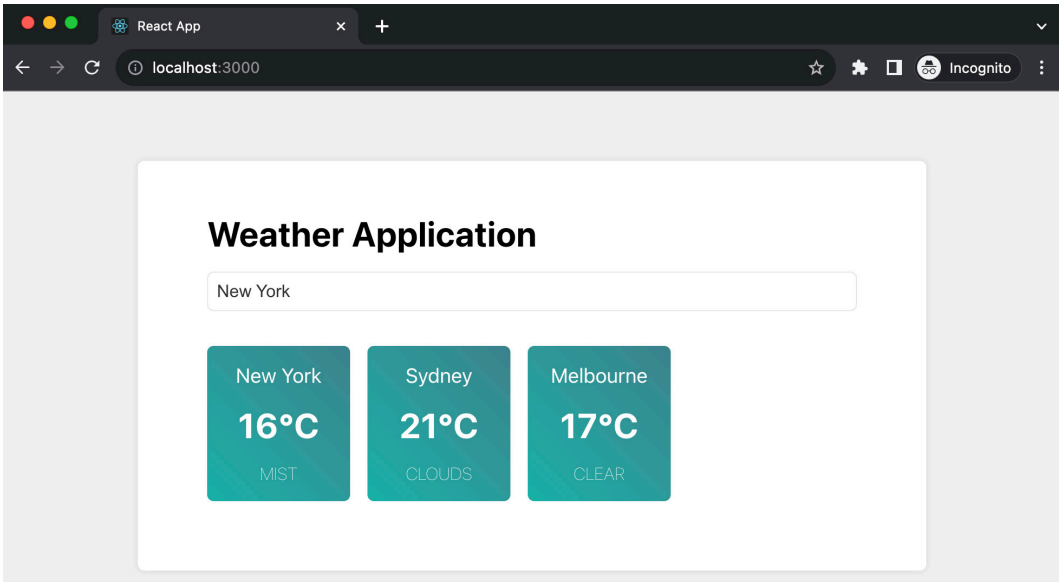

Figure 12.7: Showing multiple cities in the favorite list

Before diving into our next and final feature, it's essential to make one more straightforward improvement to the existing code – ensuring adherence to the single responsibility principle.

Refactoring the weather list

The functionality is operational and all tests have passed; the subsequent focus is on enhancing code quality. Keep the TDD approach in mind: tackle one task at a time and make incremental improvements. Then, we can extract a `WeatherList` component that renders the whole city list:

```
const WeatherList = ({ cities }: { cities: CityWeather[] }) => {
  return (
    <div data-testid="favorite-cities" className="favorite-cities">
      {cities.map((city) => (
        <Weather key={city.name} cityWeather={city} />
      ))}
    </div>
  );
};
```

The `WeatherList` component receives a cities prop, which is an array of `CityWeather` objects. It iterates through this array using `map`, rendering a `Weather` component for each city.

With the new `WeatherList` component in place, `App.tsx` will be simplified to something like this:

```
function App() {
  const { cities, fetchCityWeather } = useFetchCityWeather();
  const onItemClick = (item: SearchResultItemType) =>
   fetchCityWeather(item);

  return (
    <div className="app">
      <h1>Weather Application</h1>
      <SearchCityInput onItemClick={onItemClick} />
      <WeatherList cities={cities} />
    </div>
  );
}
```

Fantastic! Now that our application structure is clean and each component has a single responsibility, this is an opportune time to dive into implementing a new (and the last one) feature to enhance our weather application further.

Fetching previous weather data when the application relaunches

For the final feature in our weather application, we aim to retain the user's selections so that upon their next visit to the application, instead of encountering an empty list, they see the cities they selected previously. This feature is likely to be highly utilized – users will only need to add a few cities initially, and afterward, they may merely open the application to have the weather for their cities loaded automatically.

So, let's start the feature with a user acceptance test using Cypress:

```
const items = [
  {
    name: "Melbourne",
    lat: -37.8142,
    lon: 144.9632,
  },
];

it("fetches data when initializing when possible", () => {
  cy.window().then((window: any) => {
    window.localStorage.setItem(
      "favoriteItems",
      JSON.stringify(items, null, 2)
    );
```

```
  });

  cy.intercept("GET", "https://api.openweathermap.org/data/2.5/
   weather*", {
    fixture: "melbourne.json",
  }).as("getWeather");

  cy.visit("http://localhost:3000/");

  cy.get('[data-testid="favorite-cities"] .city').should("have.
   length", 1);

  cy.get(
    '[data-testid="favorite-cities"] .city:contains("Melbourne")'
  ).should("exist");
  cy.get('[data-testid="favorite-cities"] .city:contains("20°C")').
   should(
    "exist"
  );
});
```

The `cy.window()` command accesses the global window object and sets a `favoriteItems` item in `localStorage` with the items array. Following that, `cy.intercept()` stubs the network request to the OpenWeatherMap API, using a fixture file called `melbourne.json` for the mock response. The `cy.visit()` command navigates to the application on `http://localhost:3000/`. Once on the page, the test checks for one city item in the favorite cities list, verifies the presence of a city item for `Melbourne`, and confirms it displays a temperature of `20°C`.

In other words, we set up one item in `localStorage` so that when the page loads, it can read `localStorage` and make a request to the remote server, just like what we do in `onItemClick`.

Next, we'll need to extract a data fetching function within `useFetchCityWeather`. Currently, the `fetchCityWeather` function is handling two tasks – fetching data and updating the cities' state. To adhere to the single responsibility principle, we should create a new function solely for fetching data, leaving `fetchCityWeather` to handle updating the state:

```
export const fetchCityWeatherData = async (item: SearchResultItemType)
=> {
  const response = await fetch(
    `https://api.openweathermap.org/data/2.5/weather?lat=${item.
latitude}&lon=${item.longitude}&appid=<api-key>&units=metric`
  );
  const json = await response.json();
  return new CityWeather(json);
};
```